

Programming Language Design

Module 7: Parsing

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Last time

We got more comfortable with practical coding in Haskell using functional operators.

A lot of operators... (but we *love* that, right?)

This time

We're going to learn to parse.

This is incredibly important, and once you understand it, you will be most of the way to making your own toy language (i.e., our lisp dialect)

We've seen the term before, but what does it mean? [What is parsing in computer science?]

To parse

In computer science, "to parse" means "to convert a string into a value, tree, or other data structure" (my definition)

So `parseInt` in javascript converts a string to an int.

But we don't want to just stop with simple values, we want to parse *programs*.

That means we need to parse a more complex datastructure. How can we represent a *program*?

Programs are trees

The most common datastructure we use to represent a program is a tree.

Why? Consider some C code:

```
#include <stdio.h>
#include <stdlib.h>
int blort(int a, int b) {
    return 3 * a + 4 * b;
}
int main(int argc, char** argv) {
    if (argc != 3) {
        puts ("usage: blort <a> <b>");
        return 1;
    }
    printf("blort is: %d\n", blort(atoi(argv[1]), atoi(argv[2])));
    return 0;
}
```

Is that a tree?

It doesn't look like it at first glance, but a parser is turning that code into a tree.

The parser is part of the compiler. The compiler runs in several phases:

- Pre-processing (expanding `#include` and friends)
- Lexing (i.e., *lexical analysis*)
- Parsing (what we're going to cover today)
- Semantic analysis (i.e., checking types and other rules and finding errors)
- Optimization (happens in several places)
- Codegen (i.e., actually generating the lower-level code, such as assembly or llvm or machine code. We don't do this in this class: take the course in Compilers!)

Compilation units

In C, the basic unit of compilation is the...uh...*compilation unit*.

When we run the compiler, it interprets its input as a giant text file (after all the preprocessor stuff has been included) that describes a bunch of functions and variables that need to be compiled together.

The (normal) *output* of a C compiler is an object file. That is, a `.o` file.

[Whats in an `.o` file?]

Object files

You can see for yourself with the `objdump` command on *unixy* platforms. Or check the [file format](#) (it's similar on Windows but).

But basically, it's broken into "sections" of binary data:

- All the data known at compilation time goes in the `.data` section.
- All the code goes in the `.text` section. (why not call it `.code` ? I don't know)
- All the data that has space reserved at compilation time but whose size we don't know goes in the `.bss` section. (another strange name)
- A string table `.strtab` , which holds a list of exported function names (like "main")
- A symbol table `.symtab` which maps the string table strings to virtual addresses (i.e., the virtual address of the function `main`)

Where is the tree?

Buried between the input (C code) and the output (an `.o` file) is the parsing step.

We actually represent the compilation unit as some kind of tree (or forest).

Why? Because even though a C program is just a string of code, it is conceptually "structured". Specifically, structured like a tree.

Here's how our C program above could be represented as a forest...

Our program from earlier (as a prefix ordered tree)

Leaving out types:

```
compilation-unit [  
  <stuff from header files>;  
  function blort [  
    params: a, b  
    body: [  
      return [  
        + [ * [3; a]; * [4; b] ]  
      ]  
    ]  
  ];  
  function main [ ... ]; etc.  
]
```

Why?

Every function is itself a tree.

In fact, individual expressions are trees. Notice how we represent `3 * a + 4 * b`:

```
+ [  
  * [ 3, a ];  
  * [ 4, b ]  
]
```

The root is `+`, the two inner nodes are `*`, and the leaves are 3, `a`, 4, and `b`.

By using trees, we can quickly answer questions like "I want to add here, but what am I adding? Oh, it's 3 and `a`"

By turning expressions into trees, we can either execute them (i.e., interpret them), or turn them into machine code easier.

Example

Suppose we want to execute `+ [ * [3; a]; * [4; b] ]`

We can recursively execute its children and add them.

We execute `* [3, a]` and get a result.

We execute `* [4, b]` and get another result.

We add those results together.

What about generating code? It's more complicated, but it's similar. We emit code to compute `3 * a` and, e.g., push the result. We emit code to compute `4 * b` and push the result. Then we emit code to add the top two stack elements together.

(in practice we would probably not use the stack here, but this is just conceptual)

Knowledge check 1

1. Generate a tree using our goofy notation for the expression `2 + 2`
2. Now generate a tree for `2 + 2 + 2`
3. Now generate a tree for `2 + 2 * 3` but use order of operations (so the `*` would need to happen first)

KC 1 answers

1. `+ [2; 2]`
2. Several answers: `+ [2; 2; 2]`, `+ [ + [ 2; 2 ]; 2 ]`, and `+ [2; + [2; 2]]`. `+` is associative, so we don't need to consider `2 + 2 + 2` distinct from `(2 + 2) + 2` or `2 + (2 + 2)`. All are the same expression.
3. This one only has one answer: `+ [2; * [2; 3]]`

Questions?

How to parse

Assume you accept my premise that code is easier to work with (meaning execute or compile) when it is structured as a tree.

How do we actually build one of these trees?

There are a ton of ways to do it, but there is a particularly elegant way that works perfectly in Haskell called *Recursive Descent*.

Let's learn how it works...

Parsing arithmetic expressions

Let's say I want to parse an arithmetic expression like `2 + 3 * x - 1`

There are a couple of things that make it harder:

1. We need to handle in-order operations. It's not in a convenient form for turning into a tree.
2. We need to handle precedence. We want the expression to parse the same as this:

`2 + (3 * x) - 1 .`

First, let's look at what our input will actually look like...

It will be tokenized

We already talked about tokenization, so let's assume that we have tokenized this expression. It will look something like this:

```
[IntLit 2, Plus, IntLit 3, Times, Var "x", Minus, IntLit 1]
```

Assuming we have a token datatype that looks like this:

```
data Token =  
    IntLit Integer  
    | Plus  
    | Minus  
    | Times  
    | Var String  
    | etc.
```

The goal

The goal (here) is to parse a single expression. So some string of operators and operands that will end up being evaluated.

In our example, the type is simple: it's either addition, subtraction, or multiplication:

```
data Expr =  
    Add Expr Expr -- a sum (e.g., a + b)  
  |  
    Sub Expr Expr -- a difference (e.g., a - b)  
  |  
    Mul Expr Expr -- a product (e.g., a * b)  
  |  
    Val Integer -- a value by itself (e.g., 7)
```

This datatype is actually a tree. `Val` is a constructor for leaves. Otherwise, the other constructors take two expressions. This will be the result of our parsing.

`Add Expr Expr` means "this expression is the sum of two smaller expressions.", etc.

The goal (2)

We will start with some string like `2 + 3 * x - 1`, and we want to turn it into an `Expr`.

Why do we need to do that? Why not just turn it into a single integer like `7`?

Because of the `x`. We don't actually know what value this expression is going to have until we decide what `x` is.

Therefore, we have to store the whole expression, we can't just reduce it down to a single value (although if you could, feel free to try; it's a good optimization when it's available!)

The goal (3)

Therefore, we are writing this function:

```
parseExpr :: [Token] -> Expr
```

That's it. We take a string and return an expression.

The technique we use will be useful for parsing most things, including programs, not just expressions, but let's start there.

Questions before we get started?

The starting point

Okay, to start with, what *is* an expression?

Well, it could be a sum, a difference, a product, a variable, or a value.

Let's consider this as a list ordered by *precedence*:

```
expr ::= expr + term
expr ::= expr - term
expr ::= term
expr ::= term * factor
term ::= factor
factor ::= Variable
factor ::= Value
```

Backus-Naur form

That is a language that is used to define grammars called "Backus-Naur form" or (BNF).

You've probably seen it in your automata class, but here's a refresher:

BNF works like this: `production ::= rules`

This says that we can replace a list of `rules` with a production named `production`.

So if we have an expression, followed by a `+` and then a term, we can invoke this rule to combine them all into an expression:

```
expr ::= expr + term
```


Backus-Naur warnings

We have to be careful when we write our grammar.

This is not the right grammar:

```
expr ::= expr + expr  
expr ::= expr - expr  
expr ::= expr * expr  
expr ::= Variable  
expr ::= Value
```

[What's wrong with it?]

It's ambiguous

The first issue with that grammar is that it's ambiguous.

We don't just care *whether* a string is an expression, but *how*.

For example, which tree is correct for `1 + 2 * 3`:

1. `1 + (2 * 3)` which results in `Add 1 (Mul 2 3)`
2. `(1 + 2) * 3` which results in `Mul (Add 1 2) 3`

We know that we want to follow normal order of operations, which means #1. But the grammar doesn't require that, it's possible to apply rules in either order...

It's ambiguous (2)

When we see `1 + 2 * 3`,

We can apply the `expr ::= value` rule to get this: `Value 1 + Value 2 * 3`.

Then we apply `expr ::= expr + expr` to get this: `(Add (Value 1) (Value 2)) * 3`

Then we apply the `expr :: value` rule again... `(Add (Value 1) (Value 2)) * (Value 3)`

Finally, `Mul (Add (Value 1) (Value 2)) (Value 3)`

But, this is what we *want*:

Apply `expr ::= value` to `2` and `3`: `1 + Value 2 * Value 3`

Then `Value 2 + (Mul (Value 2) (Value 3))`

Then `Add (Value 2) (Mul (Value 2) (Value 3))`

So we need to make sure there's only one way to parse it.

Fixing it

We need to create intermediate categories like *term* and *factor*:

```
expr ::= expr + term
expr ::= term
term  ::= term * factor
term  ::= factor
factor ::= variable
factor ::= value
```

Now we only have one way to parse: `1 + 2 * 3`. What rules can we apply?

Previously we went bottom-up, replacing words as quickly as possible.

Instead, let's start from `expr` ...

Top-down parsing

1 + 2 * 3

There are two ways to build an expression. Either from a `+` expression, or from a term. We see a `+`, so let's try that: `expr 1 + term (2 * 3)`

We need to parse 1 as an expression and `2 * 3` as a term.

The only way to parse 1 as an expression, is to treat it like a term, which means to treat it as a factor, and then to make it a value.

To parse `2 * 3`, we invoke `term * factor`, and then parse 2 as a factor and then a value. 3 is already a factor, so we treat it as a value.

So we end up with `Add (Value 1) (Mul (Value 2) (Value 3))`

And importantly: we can't get anything else. Any other parse fails.

Associativity is fixed, too

Notice how we have this in our language:

```
expr ::= expr + term  
expr ::= expr - term  
...
```

These are left-recursive rules. What would change if we did this?

```
expr ::= term + expr  
expr ::= term - expr  
...
```

[?]

The associativity would change

If we did that, we would end up parsing $1 + 2 + 3$ as $1 + (2 + 3)$ instead of $(1 + 2) + 3$. Is that a problem?

Not for $+$, but it is for $-$: $1 - 2 - 3$ should be 4, not $1 - (2 - 3) == 1 - (-1) == 2$

So we can't do $\text{expr} ::= \text{expr} + \text{expr}$ because we get ambiguous parses

We can't do $\text{expr} ::= \text{term} + \text{expr}$ because the parentheses go around the recursive parse, and we want $+$ (and $-$) to be left recursive.

Instead, we do $\text{expr} ::= \text{expr} + \text{term}$ to get the right associativity, and to also ensure that multiplication happens before addition, even if it's on the right of a $+$.

Knowledge check 2

1. Parse $1 + 2 * x * 3 - 9 - 2$ by hand. What Haskell `Expr` do you end up with?
2. Extend the grammar by adding `^` to it (exponentiation). Make it be right associative.

If you're wondering how we extend our intuition about how to do this to Haskell, don't worry. That's coming up.

KC 2 answers

1. I like to start by putting in parentheses:

```
((1 + ((2 * x) * 3)) - 9) - 2
```

Now start putting in constructors:

```
((Add 1 ((2 * 4) * 3)) - 9) - 2
```

```
((Add 1 ((Mul 2 4) * 3)) - 9) - 2
```

```
((Add 1 (Mul (Mul 2 4) 3)) - 9) - 2
```

```
(Sub (Add 1 (Mul (Mul 2 4) 3)) 9) - 2
```

```
Sub (Sub (Add 1 (Mul (Mul 2 4) 3)) 9) 2
```

Lastly, fill in the values and variables:

```
Sub (Sub (Add (Value 1) (Mul (Mul (Value 2) (Var "x"))) (Value 3))) (Value 9)) (Value 2)
```

This is what `parseExpr` would return.

KC 2 answers (2)

```
expr ::= expr + term
expr ::= expr - term
expr ::= term
term ::= term * factor
term ::= factor
factor ::= factor ^ exp
factor ::= exp
exp ::= Variable
exp ::= Value
```

Questions?

Putting this into Haskell

Now that we have refreshed on how to use grammars, let's turn this into Haskell.

Our goal is to write this function

```
parseExpr :: [Token] -> Expr
```

However, that function is going to need to parse terms, and terms will need factors. So let's show the whole family:

```
parseExpr :: [Token] -> Expr
parseTerm  :: [Token] -> ([Token], Expr) -- the tuple has the left-over string
parseFactor :: [Token] -> ([Token], Expr)
parseVariable :: [Token] -> ([Token], Expr)
parseValue  :: [Token] -> ([Token], Expr)
```

Variables and values

Let's start with the easiest ones, `parseVariable` and `parseValue`.

We just take a `Value` token and wrap it in a `Val Expr`:

```
parseVariable (Sym name) : remainder = (remainder, Variable name)
parseVariable _ = error "expected a symbol"
```

This is why so many error messages say "expected 'blah'". It's because once we've decided we need a variable, if there isn't one there, that's an informative thing to say (it tells the user what state the parser was in).

[do `parseValue`]

Parsing factors

Parsing factor is a little more complex, but not really.

Remember the rules look like this:

```
factor ::= variable  
factor ::= value
```

So the function just has two definitions, one for a symbol and one for a literal:

```
parseFactor :: [Token] -> ([Token], Expr)  
parseFactor (Sym s) : rem = parseVariable $ (Sym s) : rem  
parseFactor (IntLit i) : rem = parseValue $ (IntLit i) : rem  
parseFactor _ = error "parsing factor: expected symbol or literal."
```

What to notice so far

So far, notice that for each rule, we have a function definition.

```
factor ::= Variable becomes parseFactor (Sym s) : rem = ...
```

```
factor ::= Value becomes parseFactor (IntLit i) : rem = ...
```

We're effectively treating our grammar rules as functions.

It's worked so far, let's keep going...

Parsing terms

The first tricky one is when parsing a term. We have two possibilities:

1. The term is just a factor (i.e., there is no `*`)
2. The term is an actual multiplication of two operands.

Let's apply the grammar and see what happens:

```
parseTerm tokens =  
  let (remaining, first) = parseTerm tokens  
  in if null remaining then ([], first) -- no operator  
     else let (remaining', second) = parseTerm' remaining  
           in Mul first second  
  where  
    parseTerm' (Times : remaining) = parseFactor remaining  
    parseTerm' _ = error "expected '*'"
```

Let's understand this first, and then try to see a problem...

Parsing terms

We are attempting to apply the parsing rule `term ::= term * factor` directly

That means, we first parse a term recursively. Then we check for a `*`. If we find one, we parse the remaining factor.

The result is either a `Term` or a `Mul (Term ...) (Factor ...)`

Either way, we return it and the rest of the tokens.

But there's a problem here. This code won't work.

[What's wrong?]

Left-recursive grammar

The problem is that the grammar is left-recursive.

So our first recursive call is unguarded.

It will therefore run forever.

This part...

```
parseTerm tokens =  
  let (remaining, first) = parseTerm tokens  
  in ...
```

...is the issue. Specifically that parseTerm call.

What's the problem?

Suppose it were right-recursive: `term ::= factor * term`

This would no longer have the associativity we want, and it would pose a problem if we added division to the language, but let's just pretend that was the rule.

Then, our function would look like this:

```
parseTerm tokens =  
  let (remaining, first) = parseFactor tokens  
    h : remaining' = remaining  
  in if h == '*' then  
    let (remaining'', second) = parseTerm tokens  
    in (remaining'', Mul first second)  
  else (remaining', first)
```

This version does not have the problem.

How can we fix it?

We have a few options:

1. Change the grammar to eliminate the recursion
2. Modify the behavior (the semantics) so that the math works out with right recursion.

Let's consider both options.

Changing the grammar

If we remove the recursive term by expanding it in the grammar (meaning, replace it with equivalent things that aren't recursive), we can get around this.

So instead of this:

```
term ::= term * factor  
term ::= factor
```

We replace the "some kind of term and a star" with this regular expression style term:

```
term ::= (factor '*' )* factor  
or  
term ::= factor ('*' factor)*
```

What's that?

That first star '*' is just the star symbol for multiplication.

That second star is a Kleene star. Just like a regular expression, it means "zero or more of the thing before me".

This modification of Backus-Naur form is called [Extended Backus-Naur form](#).

Why is it better? Well, let's revisit what it looks like to parse a term...

Parsing a term

```
term ::= factor ('*' factor)*
```

Now we just parse a factor, and then, if there's a star, we parse it, and then keep parsing factors as long as there's more.

One way of coding this is to make a recursive function that parses the rest of the term (the part starting with the first '*')...

(consider this as psuedo code to illustrate the concept: the real parsing code you'll need to use is later in this module)

Parsing a term (2)

```
-- parse a (factor '*')* into a multiply expression
parseTerm tokens =
  let (rest', factor) = parseFactor token
      (head : rest'') = rest'
  in if not (null rest'') && head == Times then
      let (rest''', factors) = parseTerm rest''
      in (rest''', Mul factor factors)
  else (rest', factor)
```

Here, we parse a factor (which we "know" needs to be there if we want to parse a term)

Then we check to see if there's any tokens left. If see, if there's also a "*" there, we keep parsing whatever comes after it.

Note: the reason it's safe to destructure the list with `(head : rest'') = rest'` before we know if it's empty or not is because of lazy evaluation.

The basic pattern

The basic pattern here is that we call a function whenever we want to match a particular grammar rule.

So there's a `parseFactor`, a `parseTerm`, and a `parseExpr`. I'll leave `parseExpr` up to you, or we can do it together if there's time.

Do we understand how we can represent rules in a grammar as functions?

This technique is called recursive descent.

Questions?

Parsing lisp

Now we get to the part that's relevant for our project.

We want to parse a Lisp program (technically our little toy version of Lisp, called "slisp").

What does the grammar look like?

Let's look at the building blocks of our language and see if we can guess the rules...

Lists

The most important datatype in Lisp is the list.

But a list of what? It can't just be lists all the way down.

(I mean, lambda calculus is functions all the way down but that's *weird*)

Like, at some point it needs to be a list *of* something, right?

We call the smallest something an *atom*.

Atoms

In a traditional lisp, there are many kinds of atoms. We will consider exactly two:

1. Symbols: some word that can be a variable name or something
2. Integers: literally an integer

So `hello` and `123` are both atoms.

What does `hello` mean? Well, it could be variable, or it could just be a random symbol. Symbols are like strings, but we don't consider them to have "characters". They are only equal to themselves.

The purpose of symbols is to represent things.

Values

Therefore, in our simple version of list, there are actually two kinds of values

1. Atoms
2. Lists of some number (including zero) of values

So there are basically 3 data-types in the whole language: symbols, integers, or lists of anything (including lists of lists of anything)

A list is delimited by parentheses `()` and the values inside are whitespace separated if they are atoms.

Representing values

What is a value as a data type? Well, something like this makes sense:

```
data Value =  
    VSym String  
  | VInt Integer  
  | VList [Value]  
  deriving Show
```

I'm adding 'V' before the constructors to distinguish them from token constructors.

Notice the `deriving Show` which we learned about from our required reading. This makes it so that we can convert our values into strings, which is useful for debugging.

List examples

The following are all valid lists

```
()  
(a b c)  
(1 2 3)  
(a b c 1 2 3)  
(a 1 b 2 c 3)  
(a (b c) 1 2 (3))
```

The symbols will eventually be interpreted as variables.

Given these examples, can we come up with a grammar that will recognize these lists?

List grammar

```
value ::= atom | list  
list  ::= '(' value* ')'  
atom  ::= Symbol | IntLit
```

So a value is either a list or an atom.

A list is some sequence of values within a pair of parentheses.

An atom is either a symbol or an integer literal

We handle symbols and integer literals with our lexer, so we stop there (no need to distinguish which is which; it's already done for us).

Note, lisp expressions are entirely prefix, so there's no need to deal with precedence or associativity. It's quite nice to parse.

Parsing that

With that grammar, let's imagine how to complete our project:

```
parseValue :: [Token] -> ([Token], Value)
parseList  :: [Token] -> ([Token], Value)
parseAtom  :: [Token] -> ([Token], Value)
```

Your job will be to implement these functions.

Think about how to do it:

- Parsing a value means determining if there's a list there (how?).
 - If there is, parse the list
 - Otherwise, it's some kind of atom. Those are easy to parse.

Suggestions

I recommend starting with `parseAtom`. You will need to either return a symbol or an integer. This can be done easily by looking at the token you've gotten and considering if it's an `IntLit`, a `Sym`, or something else.

Then `parseList` is a good next choice. It can first expect a `(`, then keep calling `parseValue` until there is a `)` token.

`parseValue` can just check if there is a `(`, and if so, call `parseList`, and if not, call `parseAtom`.

But wait, we hadn't defined `parseValue` before calling it inside `parseList`. How is it okay for `parseList` to call `parseValue`?

Mutual recursion

This is an example of **mutual recursion**.

You're used to regular recursion: make the function call itself.

Mutual recursion is when function A calls function B, but function B calls function A.

For example, consider a forest of trees, but the trees can have multiple children (so the children of a tree is a forest).

Forests and trees

```
data Tree a = Empty | InnerNode (Forest a)
data Forest a = ForestOf [Tree a]
```

A tree is either empty, or it's an inner node which has a forest for its children.

A forest is a list of trees.

They both refer to each other. This is fine. This is mutual recursion.

Practice

Back to parsing lists and atoms, What if strings were in our language? What would change about our grammar?

A single program is just a list. What if we allowed multiple lists side by side. How would the grammar change? That is:

```
((a 1 2) (b 3 4)) ; permitted  
(a 1 2) (b 3 4) ; currently not permitted. What would need to change?
```

What would change if we wanted a program to be a single atom in addition to a list?

More practice

Write a function that counts the number of nodes in our tree above.

What about forest?

Should these functions be mutually recursive?

Knowledge check on recursive descent

Consider this language, call it "Florp":

```
program ::= (statement)*  
statement ::= printStatement | assignment  
printStatement ::= 'print' sum  
assignment ::= Name '=' sum  
sum ::= value ('+' sum)*  
value ::= Int | Name
```

First, what are some example programs in "florp"?

RD Knowledge check answers (1)

Here's one:

```
x = 2
y = x + 2 + 2
z = 1 + x + y + z
print z
print x + y
```

Uh...and here's another one. Its *syntax* (order of string) is correct, but its *semantics* (what it actually means) is wrong:

```
print x
x = 2
```

Next, what should the data types be that stores the tree?

RD Knowledge check answers (2)

There are many ways to do this. One is to give each rule its own datatype.

We didn't do this for our lisp dialect because we didn't need to, but let's see what it looks like when we do:

```
data FlorpProgram = FlorpProgram [FlorpStatement]
data FlorpStatement =
    StatPrint FlorpPrint | StatAssign FlorpAssignment
data FlorpPrint = FlorpPrint FlorpSum
data FlorpAssignment = FlorpAssignment String FlorpSum
data FlorpSum = FlorpSum [FlorpVal]
data FlorpVal = FlorpInt Integer | FlorpName String
```

With this data type, we can compile an entire "florp" program into a FlorpProgram value. Then, later we can execute it. What should tokens look like?

RD Knowledge check answers (3)

Here are the important tokens:

```
data FlorpToken =  
    Print  
    | Eq  
    | Plus  
    | TInt Integer  
    | Name String
```

Practice: how would you write the tokenization routine?

What about the recursive descent functions? What would they look like?

RD Knowledge check answers (4)

Well, every florp program is a list of statements

```
parseProg :: [Token] -> ([Tokens], [FlorpStatement])
parseProg (Print : toks) =
    let (rest, printStatement) = parsePrint (Print : toks)
        (rest', otherStatements) = parseProg rest
    in (rest', printStatement : otherStatements)
parseProg (name : Eq : toks) = -- an assignment statement
    let (rest, assignmentStatement) = parseAssign (name : Eq : toks)
        (rest', otherStatements) = parseProg rest
    in (rest', assignmentStatement : otherStatements)
```

If you'd like some more practice, try the others. We have a program here, what does `parsePrint` or `parseProg` look like?

Questions?

What is the point of parsing lists?

You might be wondering what the point is. Why are we parsing lists? What's so special about them?

We actually interpret the lists to be commands. The first element of the list is a function to call, the rest of the values are arguments.

So a typical hello-world program could look something like this

```
(print "hello world")
```

Of course, that would require strings, which we don't have, but it's not hard to imagine.

Why on earth would a programming language work like this, though?

The program is an AST

Our goal with parsing is to build an abstract syntax tree, or AST.

With lisp, that is very easy, because the program itself is an AST.

Normally, we'd have to turn `print (1 + 2)` into `(print (+ 1 2))`, but with lisp, the programmer does that for us.

Once we have the AST fully turned into a Haskell tree, we can *execute* it.

Executing the AST

How does execution work?

This will be for a future lecture, but basically, our program will be a list.

We will lookup the first element of the list. If it's a `+`, we'll sum the rest of the list.

If there are any nested lists, we'll execute those.

This is how we'll make a lisp interpreter. It's a strategy that works for almost any programming language.

Questions?

Other parsing techniques

We've seen recursive descent parsing now. It's an elegant technique.

However, it can be a pain to deal with many common situations:

- Left-associative operators are annoying.
- We need to be able to differentiate nodes. For example, for the *florp* language parser, we had to check if it was a print statement or an assignment statement by looking for the `Print` token.

The last point is important. It becomes challenging when our language has rules that look almost the same except for a token that appears much later.

Other parsing techniques (2)

C compilers typically use some variant of [LR parser](#).

LR Parsers are *bottom-up* parsers instead of top-down ones like recursive decent.

They *shift* tokens into a stack until the tokens match one of the rules. Then they *reduce* the tokens into a single node matching that rule.

Consider

```
int x = 2 + 2;
```

Other parsing techniques (3)

Here, we could shift all the tokens on the stack:

[int, x, =, 2, +, 2] until we see the ;

That's a sign that we have a complete statement.

So now we reduce 2 + 2 into (+ 2 2)

[int, x, =, (+ 2 2)]

At this point, we have enough information to describe a definition.

This technique is nice because it's fast. It can be "compiled" into a giant turning machine goto table based on a description of grammar rules. This is what tools like [Flex](#) and [Bison](#) do.

Other parsing techniques (4)

So what does Haskell use?

Haskell uses a still-different technique, called an [operator-precedence parser](#)

The basic technique works like this:

- Read tokens as long as each token is either an operand or an operator that has a lower precedence than the last operator.
- At that point, you need to reduce (meaning, combine the tokens into a node).
- Then continue.

Operator precedence example

Suppose we have an expression like this:

`2 + 5 * 3^4 + 9`

We first shift all the tokens until we get to the second `+`, because that's the first time there's an operation with a lower precedence than the previous operation:

`2 + 5 * 3^4 , + 9`

We then reduce the left expression from right to left:

`(2 + (5 * (3^4)))`

Then we continue:

`(2 + (5 * (3^4))) + 9`

Then we're done.

Operator precedence

This is an extremely simple algorithm, but it works quite well.

It has problems, though. Syntax that doesn't follow the infix operator rule requires special handling.

Haskell ends up mixing some other parsing techniques to deal with that. For example, recursive descent can be used to deal with keywords.

It works great for infix operators, but struggles with mixing prefix and postfix. Haskell deals with this by making all prefix operations have the highest level of precedence and not using postfix operations.

If you are willing to abide by these strict rules, it's a great system. It also has a side benefit...

Adding operators

Haskell's precedence parser allows you to add new operators to the language.

You can literally just define a new operator any time:

```
(<==>) :: Integer -> Integer -> Integer  
a <==> b = a * b + a
```

Now:

```
20 <==> 30 == 620
```


Adding operators (2)

You can even change its precedence:

```
infixl 6 <==> -- put this after the type declaration, before the definition
```

Now our goofy space-ship-looking operator has precedence level 6, the same as `+` and `++`. `*` is 7. `^` is 8.

The lowest precedence are `$`, `$!` and `seq`. This is why `$` is a useful substitute for parentheses (because we can guarantee everything to the right of it runs first).

[Here's the list](#). The reason this is all possible is that operator precedence parsers only require a table that maps operators to their precedence. This table isn't static, unlike the way LR parsers usually work.

Parser combinators

One last technique. Haskell is a pretty flexible language, more flexible than we've seen.

It's annoying to have to bind every parse in a `let` expression that pulls out the remaining tokens and the thing we just parsed.

There are combinators for parsing, just like for lambda expressions. These combinators are functions that allow you to say "expect this token followed by any number of these other rules".

[Here are some examples](#). Don't expect to understand the ideas until we get to monads later, but it's nice to know that parsing can be as clean as basically putting the grammar into a Haskell program.

Required reading

[Classes and Types](#) (Very important!)

[The Functor Class](#) (Also very important!)

Questions?